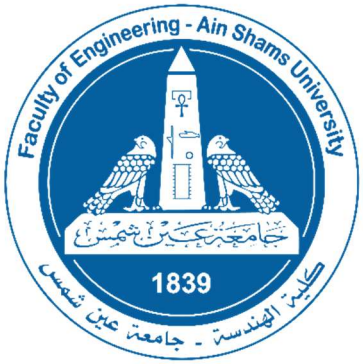


Vehicle Rental System

CSE224: Database Design - Phase 1
Team 14

MEMBERS

Adham Hisham Kandil

22P0217

Yousif Salah Mohammed

22P0232

Saifeldin tamer elsayes

22P0233

Ahmed Mohamed Talaat

22P0176

Jana Sameh Samir

22P0237

Omar Hassan Elsayed

22P0167

Supervised by

Dr. Nivin Atef
Professor, Computer and Systems Engineering

Eng. Amira Fekry
Teaching Assistant, Computer and Systems Engineering

Table of Contents

Table of Figures

1. Vehicle Rental System Database Design

This report presents the database structure for a **Vehicle Rental System**, enabling seamless interactions between customers, rental service providers, and employees. The system is designed to manage vehicle availability, customer bookings, payments, maintenance records, and reviews efficiently.

The **Entity-Relationship Diagram (ERD)** illustrates the database framework, defining entities, attributes, and their relationships. **Normalization techniques (1NF, 2NF, 3NF)** are applied to eliminate redundancy and ensure data consistency. The **Relational Schema** translates the ERD into a structured database model, optimizing data organization and retrieval.

Additionally, this report outlines key design decisions, including the roles of various entities and their interactions, ensuring a scalable and well-structured database. The system supports real-time vehicle rental processing, secure transactions, and effective maintenance tracking, enhancing overall efficiency and user experience.

2. Entity Description and Design Analysis

2.1 Customer Entity

- A **Customer,** identified by a unique **CustomerID,** represents an account that holds individual information for each customer
- A **Customer** will have a **Customer_Name**, **Address**, **Phone_Number**, and **Email**
- Each **Customer** account has a **Password** which will be stored as a hashed value in the final database
- **Customer** accounts will also have a stored **Date_of_Birth** which will be used to generate the derived **Age** attribute on the fly
- Each **Customer** will also have a composite attribute **Personal_License** which contains a **Personal_Photo**, **Expiry_Date**, and **License_Number**

Design Choices

- Having the **Age** attribute be calculated on the fly will ensure that the age will always be accurate without needing the customer to update it
- Storing the **License** for each **Customer** allows for hassle-free rentals as the **Customer** wouldn't have to go through license verification every time

---

2.2 Rental Entity

- A **Rental** is identified by its connection with a specific **Customer** & a specific **Vehicle** as well as a unique **RentalID**
- **Rental** entities keep track of the **Start_Date** and **End_Date** of the rental agreed upon with the **Customer**
- **Rental** will also store the current **Rental_Status** of the rental (pending, active, completed, canceled)

Design Choices

- Specific **Rental** information such as Vehicle information or Customer specific information are stored in each respective entity and can be queried when needed using the relationships between the entities

2.3 Payment Entity

- Payment is identified by the **CustomerID** & **RentalID** which allows it to be uniquely tied to a specific **Rental** made by a specific **Customer**
- Each **Payment** entity stores the corresponding **Payment_Status, Payment_Date** and the **Total_Price** paid
- **Payment_Status** indicates the current status of the payment (pending, completed, failed)
- **Total_Price** is the final price including any discounts or extra fees on the **Rental**
- **Payment_Date** stores the date the **Payment_Status** was last updated

---

2.4 Employee Entity

- An **Employee**, identified by a unique **EmployeeID**, represents an individual working for the company and ensures that every employee in the system is uniquely identifiable
- Each **Employee** has a **Name and Age** stored in the **Employee** entity
- **Salary** represents the compensation received by the **Employee** which is useful for payroll processing and financial records.
- Each **Employee** has a position within the company defined as **Role** which helps in assigning responsibilities and managing permissions within the system
- **Enrollment_Date** records the joining date of the **Employee**.

2.5 Reviews Entity

- **Review** is identified by the **VehicleID** which allows it to be uniquely tied to a specific **Vehicle** being reviewed which helps in tracking feedback for different **Vehicles**.

- **Review** is also identified by the **RentalID** which links the review to a specific **Rental** and ensures that only verified customers who have rented a vehicle can leave a review

- **Customer_Name** stores the name of the **Customer** who wrote the **Review**.

- Each **Review** has a numerical **Rating** given by the **Customer** which helps in calculating overall vehicle or service **Ratings**.

- **Feedback** is a text-based **Review** where **Customers** share their rental experience, helps in identifying areas for improvement in the rental service.

2.6 Vehicle Entity:

- Each **vehicle** is identified by its unique **VehicleID.**
- Each car has its **brand**, **model** (**Name** and **Year** of manufacturing), vehicle **type**, **Number_of_seats** and **color** which could be more than one color.
- Each vehicle should have its **license** details like **License_Number** and **Expiry_Date**.
- The system should store the **Availability_Status** of the vehicle (Available, Rented or In Maintenance).

2.7 Garage:

- Each **garage** should be identified by its unique **GarageID.**
- The **Garage_name** and the **manager_name** should be stored.
- The **address** of the garage should be stored (**State** and **City**).
- Each garage should have its own **phone_number**.

2.8 Maintenance:

- Each **maintenance** service done to a vehicle should have a **maintenance_number**.
- The maintenance **company_name** should be stored.
- All the **service_prices** should be stored.
- The **date** of each service should be stored.

3. Relationships

3.1 Customer – Views – Vehicle (M: N):

- A **Customer** can view multiple **Vehicle**s, and each **Vehicle** can be viewed by many customers.
- This is a many-to-many relationship and allows **Customer**s to see the current catalog of **Vehicle**s they can rent

3.2 Vehicle - Offered For - Rental (M: N):

- Multiple **Vehicle**s are offered for each **Rental**, and multiple **Rental**s can be made for each **Vehicle**
- A **Rental** entity is made on a specific **Vehicle** the **Customer** viewed and decided on
- This relationship holds the **Daily Price** which is the updated price of the **Vehicle** at any time

3.3 Customer – Makes – Rental (1:M):

- A **Customer** can make multiple **Rental**s, but each **Rental** is only associated with one **Customer**
- **Rental** is fully dependent on the **Customer** entity as it would not exist without being tied to a specific **Customer**.

3.4 Employee – Checks – Review (1:M):

- An **Employee** checks multiple **Reviews** left by customers, but each **Review** is checked by only one **Employee**.
- **Report** represents any notes, flags, or decisions made by the **Employee** after checking a customer's **Review**.

3.5 Rental – Includes – Review (1:1):

- Each **Rental** transaction must include exactly one **Review**, and each **Review** belongs to exactly one **Rental** transaction

- **Review** is fully dependent on the **Rental** entity as **Review**s can only be done by a **Rental**

3.6 Customer – Pays – Payment (1:M):

- Each **Payment** is made by one **Customer**, but a **customer** can have many **Payment**s
- **Payment** is fully dependent on the **Customer** entity as **Payment**s can only be done by a **customer**

3.7 Garage-Stores-Vehicle (1:M):

- **One** garage can store **multiple** vehicles, while a vehicle can be only available in one garage at a time. So the cardinality is **1:M.**
- A **vehicle** does **not** have to be **stored** in a **garage** all the time because it may be rented or undergoing maintenance, so it has **partial** participation in the Stores relationship.
- A **garage** is **not** required to store a **vehicle** at all times, meaning it can exist without having any vehicles, so it also has **partial participation** in the Stores relationship.
- The **Vehicle_count** in the Garage at the current moment is stored.

3.8 Employee-Works_For-Garage (N:M):

- **Multiple** Employee can work for a single garage, and one employee can work for **many** garages. So the cardinality is **N:M.**
- **Not all employees** are required to work for a garage, as some may have other tasks, such as checking reviews, so the Employee entity has **partial** participation in the Works_For relationship.
- **Every garage must** have at least one employee working for it, so the Garage entity has **total** participation in the Works_For relationship.

3.9 Vehicle-Needs-Maintenance (N:M):

- **One vehicle** may require **multiple maintenance** services over time, and **each maintenance** record may apply to **multiple** vehicles (for example, routine inspections for a fleet of cars). So, the cardinality is **M:N.**
- Since **every vehicle** will require maintenance at some point, the Vehicle entity has **total** participation in the Needs relationship.
- **Every maintenance** record must be linked to at least one vehicle, meaning it cannot exist independently. Therefore, the Maintenance entity also has **total** participation in the Needs relationship.

Total Relationships:

1:1 Relationships (One-to-One)
Includes (Rental ↔ Reviews)
Total 1:1 Relationships = 1

1: N Relationships (One-to-Many)
Stores (Garage ↔ Vehicle)
Pays (Customer ↔ Payment)
Makes (Customer ↔ Rental)
Checks (Employee ↔ Reviews)
Total 1: N Relationships = 4

N: M Relationships (Many-to-Many)
Views (Customer ↔ Vehicle)
Offered For (Vehicle ↔ Rental)
Needs (Vehicle ↔ Maintenance)
Works For (Employee ↔ Garage)
Total N: M Relationships = 4

Final Count of Relationships
1:1 Relationships = 1
1: N Relationships = 4
N: M Relationships = 4
Total Relationships = 9

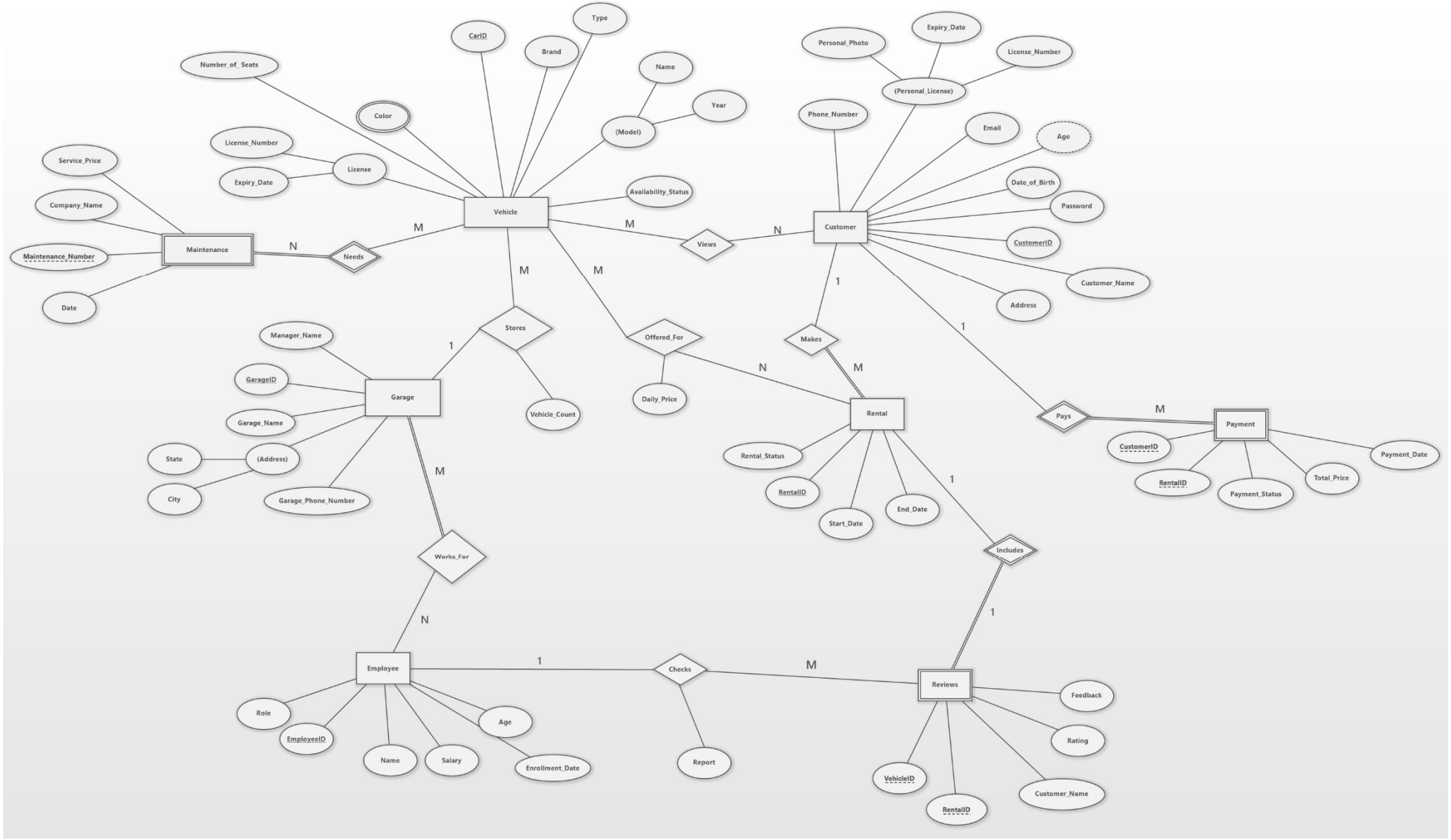


Figure 1

ER Diagram

[Diagram Picture Link]

4. Mapping EER Diagram to Relational Model

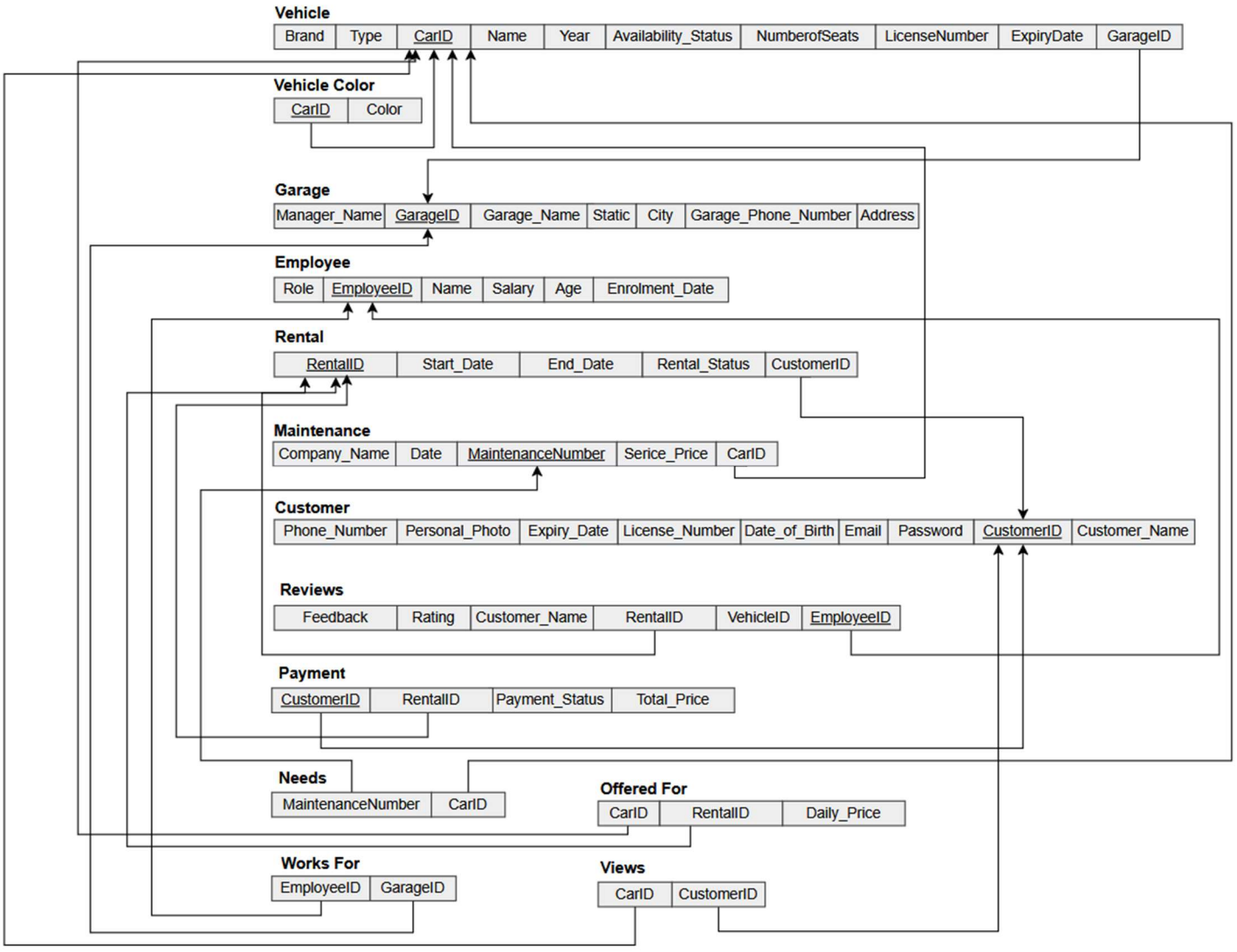
To map the EER diagram into a relational schema, we considered the following mapping rules:

• For each **1: 1** binary relationship, in the total participation entity, add the other entity's primary key as the foreign key.
• For **1: N** binary relationship, add to the entity on the N side the primary key of the other entity as the foreign key.
• For **M: N** binary relationship, make a new entity with a foreign key as the primary key of the two participating entities. Their combination forms the new primary key.
• For mapping weak entities, the primary key of the owner entity is added as foreign key in the weak entity relation.

• For each multivalued attribute, create a new relation that will include the **primary key** of the relation, the attribute is part of a **foreign key.**
• For composite attributes include the last level of the attributes in the relation.
• For Overlapping and Disjoint specializations, a new relation is created with the primary The key of this relation is the same as that of the primary key of the parent relation.

After applying these rules, the following changes were made to the mapping

- **Vehicle**: GarageID is a foreign key.
- **Vehicle_Color**: Car_id is a foreign key.
- **Garage**: No foreign keys.
- **Employee**: GarageID is a foreign key.
- **Rental**: CustomerID and Car_id are foreign keys.
- **Maintenance**: VehicleID and CarID are foreign keys.
- **Customer**: No foreign keys.
- **Reviews**: CustomerID, Rental_Number, VehicleID, and EmployeeID are foreign keys.
- **Payment**: CustomerID and Rental_Number are foreign keys.
- **Needs**: Maintenance_Number and Car_id are foreign keys.
- **Offered_For**: Car_id and Rental_Number are foreign keys.
- **Works_For**: EmployeeID and GarageID are foreign keys.
- **Views**: Car_id and CustomerID are foreign keys



Relational Schema Diagram

Figure 2

Relational Schema Link

5. Database Normalization

Normalization is a crucial process in database design that ensures data accuracy, minimizes redundancy, and enhances overall efficiency. By structuring data properly, we eliminate inconsistencies and improve query performance. Our database follows the three normal forms (1NF, 2NF, and 3NF) to maintain integrity and optimize operations.

5.1 First Normal Form (1NF)

Eliminating Repetitive Data

A database table adheres to 1NF if:

- Each column contains only atomic (indivisible) values.
- There are no groups of repeating data within a column.
- Each row has a distinct primary key.

Enhancements for 1NF Compliance:

- Introduced a **Vehicle_Color** table to store multiple colors per vehicle as separate records instead of combining them into a single column.
- Established a **Reviews** table to ensure each customer review is recorded independently rather than stored collectively in one field.
- Created the **Needs** table to manage vehicle maintenance tasks separately, ensuring atomic storage of maintenance details.

5.2 Second Normal Form (2NF)

Removing Partial Dependencies

A table meets 2NF criteria if:

- It satisfies the conditions of 1NF.
- All non-key attributes fully depend on the primary key, avoiding partial dependencies.

Modifications for 2NF Compliance:

- Separated **Rental** details from vehicle and customer information, ensuring that each entity is linked through unique identifiers instead of storing redundant details.
- Established a **Payment** table to handle transaction records independently of rental data, maintaining clear relationships through foreign keys.
- Developed an **Offered_For** table to manage vehicle availability, preventing redundant storage of rental options in the **Vehicle** table.

5.3 Third Normal Form (3NF)

Eliminating Transitive Dependencies

A database satisfies 3NF when:

- It is adhered to 2NF.
- Every column depends only on the primary key, with no indirect relationships between non-key attributes.

Optimizations for 3NF Compliance:

- Extracted **Garage** details from the **Vehicle** table, ensuring vehicle information is not dependent on garage attributes.
- Introduced a **Works_For** table to link employees with garages, preventing unnecessary duplication of garage details in the **Employee** table.
- Refined the **Customer** table by relocating payment information to the **Payment** table for better organization.

5.4 Benefits of Normalization

Implementing normalization enhances database structure by:

- **Reducing redundancy**, ensuring data is stored in a structured and efficient manner.
- **Maintaining consistency**, where updates and deletions affect only relevant records without discrepancies.
- **Enhancing performance**, allowing for quicker searches, updates, and retrieval operations.
- **Preventing anomalies**, such as insertion, update, and deletion issues that arise from poor database design.

By applying 1NF, 2NF, and 3NF, our database remains **well-structured, scalable, and optimized**, ensuring seamless management of rentals, payments, and maintenance processes.